

Homework3 CS5800

Rayan Hassan

January 2025

1 Exercise 1

(a)

We can use a max-heap to automate the process. Each node would store the severity and the arrival time of a patient. Any parent node therefore has severity greater than or equal to its children's. To find the next patient, it would only take $O(1)$ since the patient with the highest severity is at the root of the heap.

(b)

It would take $O(\log n)$ to insert a new patient. This is because we first have to add them to the end of the array, then we have to heapify to restore the structure of the heap. This means that starting from the position of the newly added patient, we compare it with its parents then if the new patient has a higher severity they get swapped. This will repeat until the new patient is in a position where its parent has a higher (or equal) severity or if the new patient becomes the root. So the total amount of time this takes is equal to the height of the tree, which is $O(\log n)$. In the same way/reasoning, it would take $O(\log n)$ to reorganize after a patient is sent to a doctor.

(c)

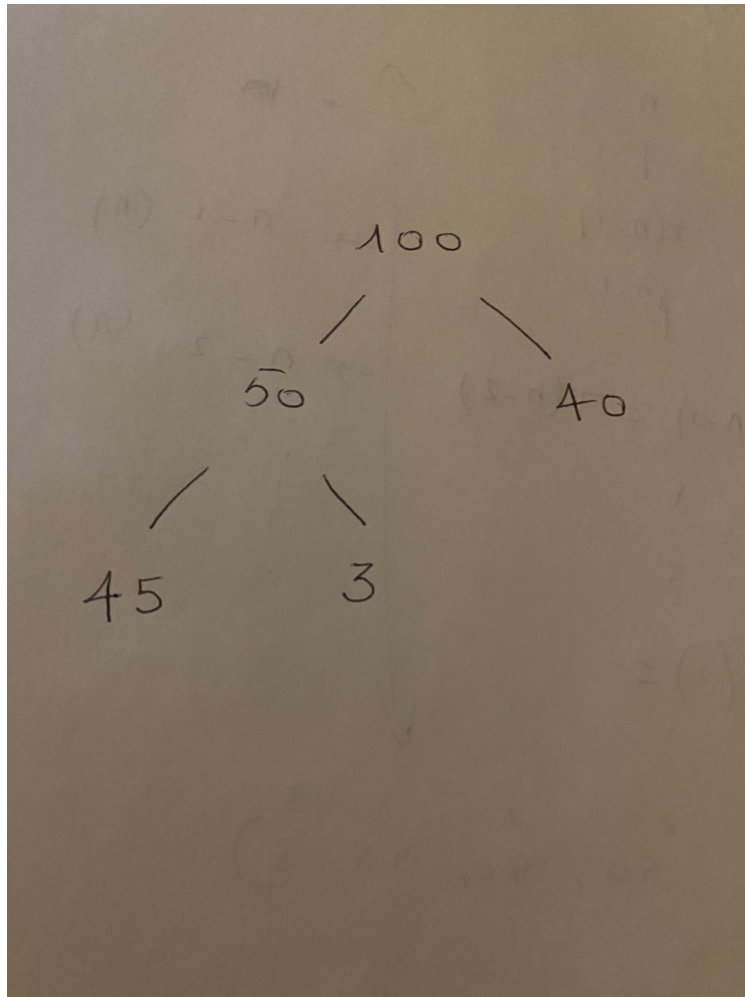
If a patient gets worst while waiting, first we have to find them (which would take $O(n)$), then we have to update their severity, and lastly we have to reinsert them into the heap (which would take $O(\log n)$ as explained before).

(d)

If the numbers are integers we can simply use an array of linked lists. We would first scan the array from 100 down to 1 to find the next patient (with the highest severity), and since now we have a fixed severity range, this would only take $O(1)$. Adding a new patient would also just cost $O(1)$ since we just need to add the patient to the queue at the array index that corresponds to their severity. And then finally, updating the severity would also only take $O(1)$. Which means that the total operation would only cost $O(1)$ which is much more efficient.

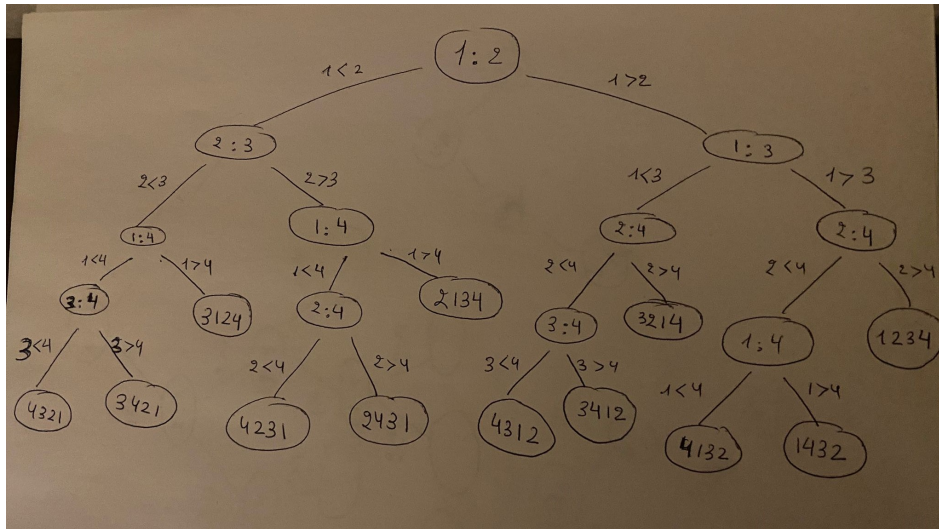
2 Exercise 2

100 is the first element (always the root) so it cannot be x. 50 is the left child of 100, it cannot be x. 40 is the right child of 100, it cannot be x. 45 is the left child of 50, it could be x. 3 is the right child of 50, it could be x. Here is how the heap looks like:

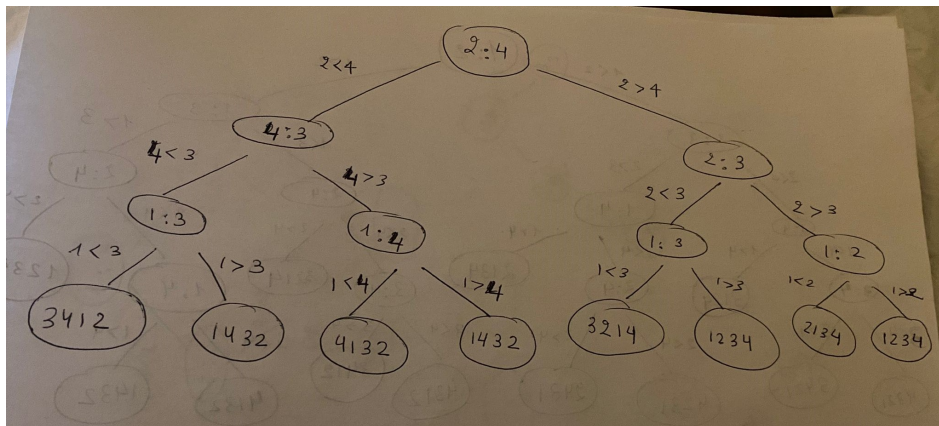


3 Exercise 3

(a)



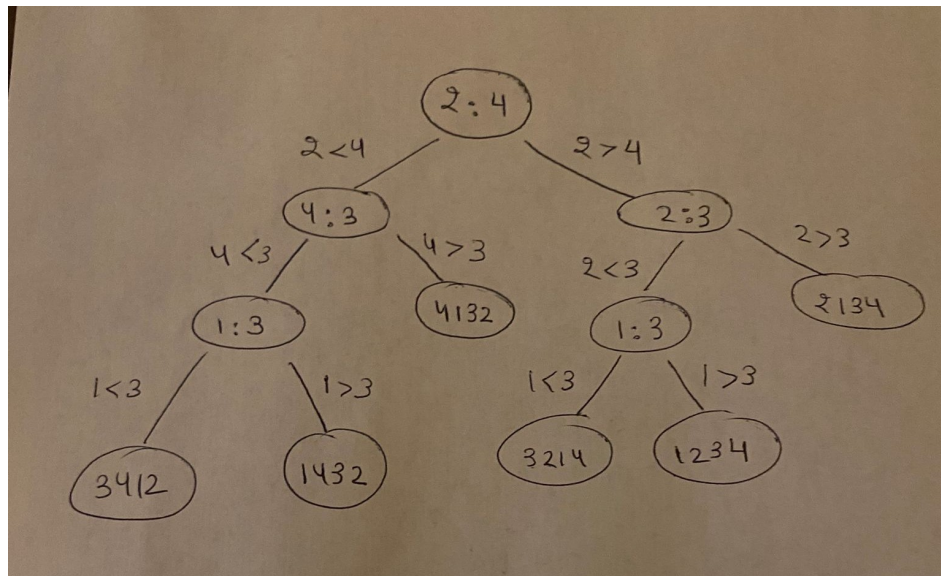
(b)



(c)

My algorithm follows the same methodology as the reverse method used in (b), but since in the tree shown there we can see that we have some duplicate leaves (specifically 1432 and 1234), this new algorithm gets rid of them. There I do not compare 1 and 4, and 1 and 2 anymore. I simply say that after comparing

3 and 4, if $4 > 3$, then the leaf is directly 4132 (without comparing 1 and 4). Similarly, after comparing 2 and 3, if $2 > 3$, I directly say that leaf is 2134 (without comparing 1 and 2). The following tree clarifies it better:



4 Exercise 4

For n elements, there are $n!$ possible permutations or inputs. The minimum height of a binary tree with $n!$ leaves is $\log_2(n!)$. So $height \geq \log_2(n!)$.

Using Stirling approximation: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$. If we take the log we get:

$$\log_2(n!) \approx n \log_2 n - n \log_2 e + \frac{1}{2} \log_2(2\pi n).$$

The dominant term here is $n \log_2 n$ here which means $\log_2(n!) = \Omega(n \log n)$. This shows that sorting n elements always takes at least $\Omega(n \log n)$ comparisons.